Supplementary Material

1 DERIVATION OF BACK PROPAGATION FOR LEAKY INTEGRATE-AND-FIRE NEURON MODELS (WITHOUT WINNER-TAKE-ALL)

Leaky integrate-and-fire neuron model in matrix form:

$$V_t = Wx_t + \alpha V_{t-1} \odot (1 - y_{t-1})$$
$$y_t = h(V_t - V_{th})$$
$$E = \frac{1}{2N}\sum_{i=0}^{N}(y_{i,t} - \hat{y_{i,t}})^2$$
$$\Delta W = -\eta \frac{\partial E}{\partial W}$$

Where V_t is membrane voltage vector in the selected layer at time t with the size of $n \times 1$ (n is the total number of neurons in this layer), x_t is the input signals fed to selected layer at time t with the size of $m \times 1$ (m is the total number of neurons in last layer), W represents the weight matrix between current current layer and the last layer with the size of $n \times m$, α is the leakage factor, $\odot$ denotes the element-wise product, V_{th} is the membrane voltage threshold, y_t is the output vector for the selected layer with only 0 and 1 to indicate if there is a spike generated with the size of $n \times 1$, and $\hat{y}_t$ is the correct output state for output layer with the same size of that of the output layer, E is the cost function calculating the distance between y_t in the output layer and $\hat{y}_t$, N is the number of output neurons, and ΔW gives the weight update matrix with the same size as that of W.

According to chain rule, $\frac{\partial E}{\partial w_{l,t}}$ for the output layer can be derived as below:

$$\begin{aligned}\frac{\partial E}{\partial W_{l,t}} &= \frac{\partial E}{\partial y_{l,t}} \cdot \frac{\partial y_{l,t}}{\partial V_{l,t}} \cdot \frac{\partial V_{l,t}}{\partial W_{l,t}} \\ &= (y_{l,t} - \hat{y}_t) \odot h'(V_{l,t} - V_{th}) \cdot x_{l,t}^T \\ &= \delta_{l,t} x_{l,t}^T \quad \textbf{let } \delta_{l,t} = (y_{l,t} - \hat{y}_t) \odot h'(V_{l,t} - V_{th})\end{aligned}$$

After that we can back propagate the error from output layer:

$$
\begin{aligned}
\frac{\partial E}{\partial W_{l-1,t}} &= \frac{\partial E}{\partial y_{l,t}} \cdot \frac{\partial y_{l,t}}{\partial V_{l,t}} \cdot \frac{\partial V_{l,t}}{\partial W_{l-1,t}} \\
&= (y_{l,t} - \hat{y}_t) \odot h'(V_{l,t} - V_{th}) \cdot \frac{\partial V_{l,t}}{\partial W_{l-1,t}} \\
&= (W_{l,t}^T \delta_{l,t} \odot h'(V_{l-1,t} - V_{th})) x_{l-1,t}^T \\
&= \delta_{l-1,t} x_{l-1,t}^T \quad \textbf{let } \delta_{l-1,t} = W_{l,t}^T \delta_{l,t} \odot h'(V_{l-1,t} - V_{th})
\end{aligned}
$$

Therefore, the weight update matrix ΔW can be expressed as follow:

$$
\begin{aligned}
\Delta W_k &= -\eta \delta_{k,t} x_{k,t}^T \\
\delta_{k,t} &= \begin{cases} \frac{1}{N}(y_{k,t} - \hat{y}_t) \odot h'(V_{k,t} - V_{th}) & \textbf{if k = K} \\ (W_{k+1,t}^T \delta_{k+1,t}) \odot h'(V_{k,t} - V_{th}) & \textbf{otherwise} \end{cases}
\end{aligned}
$$

Where K is the index of the output layer.

The step function $h(V_t - V_{th})$ is non-differentiable, therefore we use a straight-through estimator proposed by this work[1] shown as follow:

$$
h'(V_t - V_{th}) = \begin{cases} \frac{1}{2V_{th}} & \textbf{if } 0 < V_t < 2V_{th} \\ 0 & \textbf{otherwise} \end{cases}
$$

[1] Yin, Shihui & Kolala Venkataramanaiah, Shreyas & Chen, Gregory & Krishnamurthy, Ram & Cao, Yu & Chakrabarti, Chaitali & Seo, Jae-sun. (2017). Algorithm and Hardware Design of Discrete-Time Spiking Neural Networks Based on Back Propagation with Binary Activations.

2 DERIVATION OF BACK PROPAGATION FOR LEAKY INTEGRATE-AND-FIRE NEURON MODELS (WITH WINNER-TAKE-ALL)

To have winner-take-all network-level restriction to only allow at most one neuron to fire at each time step, a softmax layer is added to the previous output layer:

$$S_t = softmax(V_t \odot y_t)$$

Where S_t is a vector with the size of $N \times 1$. With the new output representation becoming continuous values, we apply cross entropy loss as the cost function:

$$E = -\sum_{i=0}^{N} \hat{y_{i,t}} ln(S_{i,t}) = -ln(S_{j,t})$$

Where j is the index of the output neuron that should fire according to the ground truth.

Therefore, the derivation of the gradient for the output layer is as below:

$$\begin{aligned}
\frac{\partial E}{\partial W_{l,t}} &= \frac{\partial E}{\partial S_t} \cdot \frac{\partial S_t}{\partial V_{l,t}} \cdot \frac{\partial V_{l,t}}{\partial W_{l,t}} \\
&= \frac{\partial E}{\partial S_t} \cdot \frac{\partial S_t}{\partial Z_{l,t}} \cdot \frac{\partial Z_{l,t}}{\partial V_{l,t}} \cdot \frac{\partial V_{l,t}}{\partial W_{l,t}} \quad \textbf{let } Z_{l,t} = V_{l,t} \odot y_{l,t} \\
&= (S_t - \hat{y_t}) \odot (y_{l,t} + V_{l,t} \odot h'(V_{l,t} - V_{th}) x_{l,t}^T \\
&= \delta_{l,t} x_{l,t}^T \quad \textbf{let } \delta_{l,t} = (S_t - \hat{y_t}) \odot (y_{l,t} + V_{l,t} \odot h'(V_{l,t} - V_{th}))
\end{aligned}$$

From the output layer results, we can back propagate the errors and get the expression for other layers as follow:

$$\begin{aligned}
\frac{\partial E}{\partial W_{l-1,t}} &= \frac{\partial E}{\partial S_t} \cdot \frac{\partial S_t}{\partial V_{l,t}} \cdot \frac{\partial V_{l,t}}{\partial W_{l-1,t}} \\
&= \frac{\partial E}{\partial S_t} \cdot \frac{\partial S_t}{\partial Z_{l,t}} \cdot \frac{\partial Z_{l,t}}{\partial V_{l,t}} \cdot \frac{\partial V_{l,t}}{\partial W_{l-1,t}} \quad \textbf{let } Z_{l,t} = V_{l,t} \odot y_{l,t} \\
&= (W_{l,t}^T \delta_{l,t} \odot h'(V_{l-1,t} - V_{th})) x_{l-1,t}^T \\
&= \delta_{l-1,t} x_{l-1,t}^T \quad \textbf{let } \delta_{l,t} = (W_{l,t}^T \delta_{l,t} \odot h'(V_{l-1,t} - V_{th})
\end{aligned}$$

Therefore, the final weight update matrix expression is as below:

$$\Delta W_k = -\eta \delta_{k,t} x_{k,t}^T$$

$$\delta_{k,t} = \begin{cases} (S_t - \hat{y}_t) \odot (y_t + V_t \odot h'(V_{k,t} - V_{th})) & \textbf{if k = K} \\ (W_{k+1,t}^T \delta_{k+1,t}) \odot h'(V_{k,t} - V_{th}) & \textbf{otherwise} \end{cases}$$

3 PSEUDO CODE FOR PULSE PARAMETER SELECTION MODULE

```
procedure PULSE PARAMETERS SELECTION ACCORDING TO ΔW
    memristor = ParametericDevice(*args)                    ▷ Create a ParametericDevice object
    res = []                                                ▷ Create an empty list to store resulting resistance
    for pulse in pulseList do                               ▷ Loop over all options of pulse parameters
        memristor.initialise(R)                             ▷ Set current resistance to R
        for timestep in range(pulse[1] / dt) do             ▷ pulse[1] stores pulsewidth
            step_dt(pulse[0], dt)                           ▷ pulse[0] stores magnitude
        end for
        res.append(memristor.Rmem)                          ▷ Append resulting resistance to the list
    end for
    resDist = abs(res - R_expected)   ▷ Calculate distance between expected and calculated resistance
    selectedPulseIndex = argmin(resDist)                    ▷ Find the index of minimal distance
    selectedPulse = res[selectedPulseIndex]                 ▷ Find parameters of the pulse option
end procedure
```

4 MAIN PANEL GUI

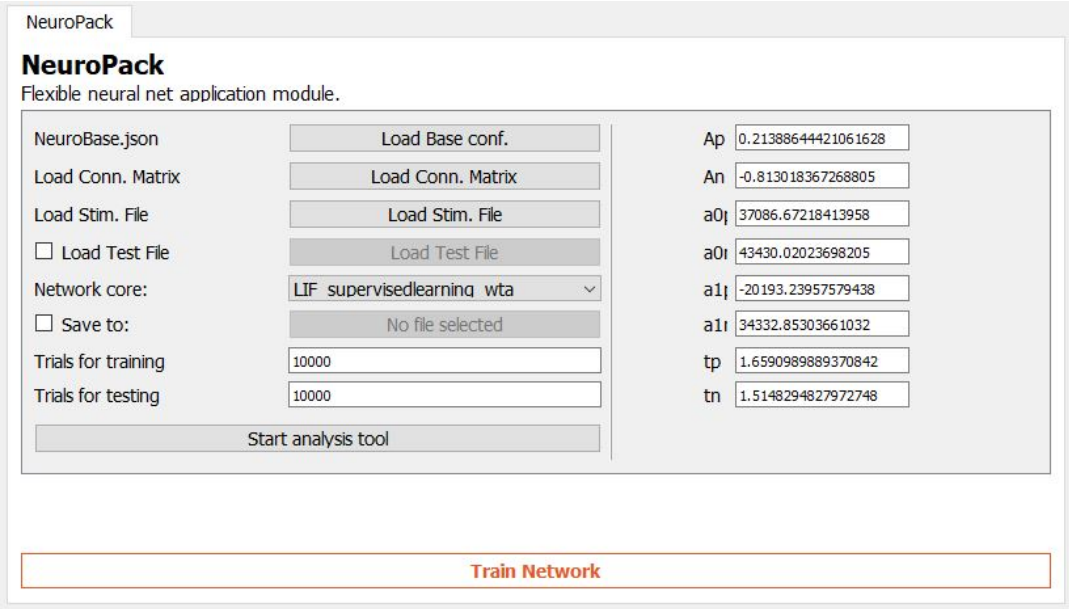

Figure S1. Screenshot of the main panel.

Figure S1 shows the screenshot of the main panel. Through the main panel, users can load input files, select the core file (the neuron model and the learning rule), and set memristor parameters. 'Network cores' gives a drop-down list showing all example and user-defined core files. By selecting one of them, the neuron model and the learning rule is set for the simulation. To execute the run, the "Train Network" button is pressed. This will generate a "run file" that contains a log of membrane voltages, weights, etc. generated during the run and can later be opened by the analysis tool for further investigation. The 'Start analysis tool' button boots said data analysis tool.

5 ANALYSIS TOOL GUI

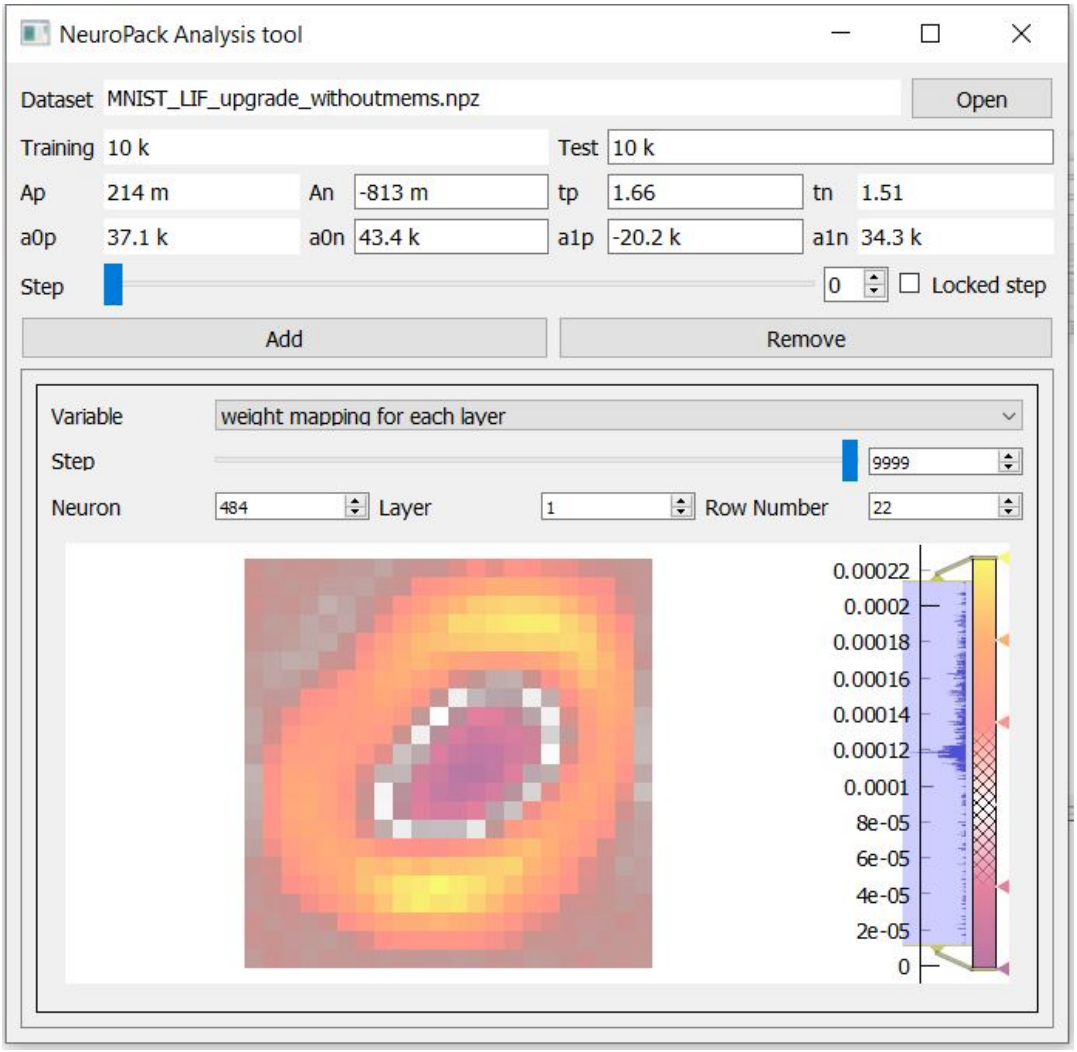

Figure S2. Screenshot of the analysis tool.

Figure S2 shows the screenshot of the analysis tool. It runs in a separate panel (from the main panel), but is invoked from the main panel, as shown in Figure S1. To run the analysis tool, users need to enable 'Save to' with a given file name to store inference and training results in the main panel. In the analysis tool, first the desired "run file" file is selected, and parameters such as training & test epochs, and memristor parameters are automatically extracted and displayed. By clicking the 'Add' button, an analysis window is added. Users can then select a variable to plot from the drop-down list (in this example the weight map for a particular neuron), which will display the corresponding images for a selected time step. Selectable variables include weight mappings, stimuli inputs, membrane voltages, fire history, etc. In this figure, we show the weight mapping for neuron 484 in layer 1 (the output layer) displayed as a rectangle with 22 pixels in a row. A weight distribution summary is also displayed to the right of the image (weights are shown in units of Siemens in the present implementation).

6 DATASET

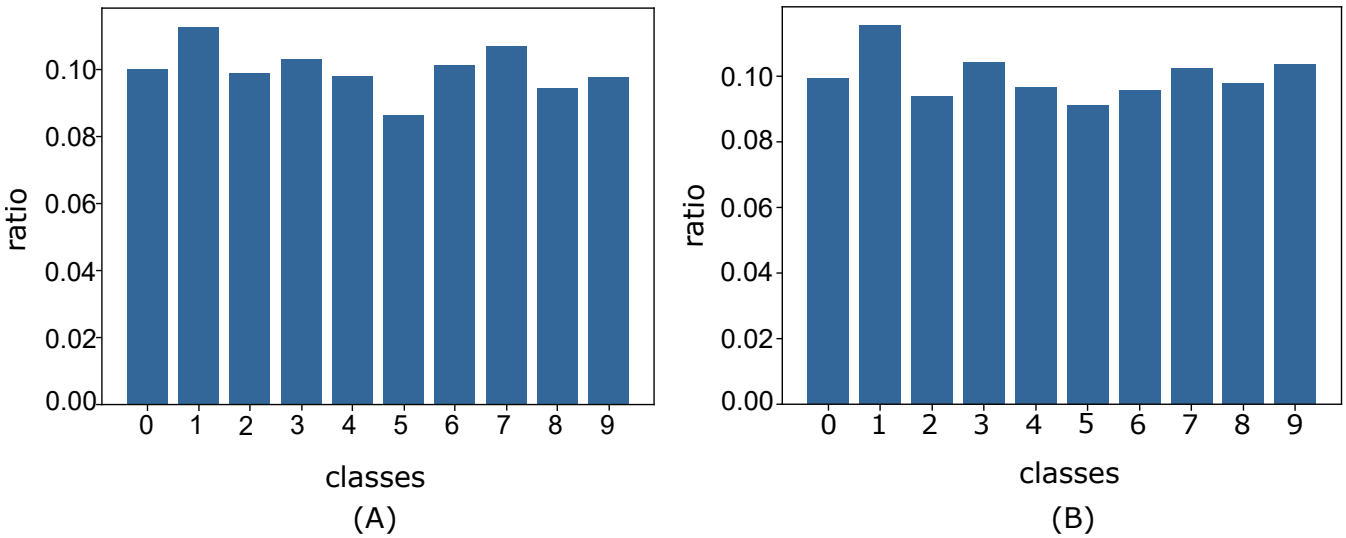

Figure S3. Proportions of digits in the whole MINST training set (A - 10k samples), and our chosen test set (B - reduced to 2k samples)

The calculated euclidean distance between the used 2k and the original 10k test sets is 0.019 using the equation below:

$$\text{Euclidean distance} = \sqrt{\sum (f_{\text{original MNIST}} - f_{\text{2k test set}})^2}$$

Where f represents statistic frequency of a digit in the whole dataset.

7 ACCURACY IMPROVEMENT

We investigated the reason why the baseline accuracy given in 'Results' part is relatively low. We tried three mitigation methods including increasing test data samples, increasing training samples, and fine tuning parameters. Figure S4 summarises what we found:

Figure S4. Accuracy comparison between using 10k test data and 2k test data for different methods.

We found a small difference between using 10k test data and 2k test data. We attribute this to the fact that the 2k test dataset is relatively small, and the slight unbalance of test data creates the difference in test accuracy for each class (shown in Figure S5) to the total test accuracy.

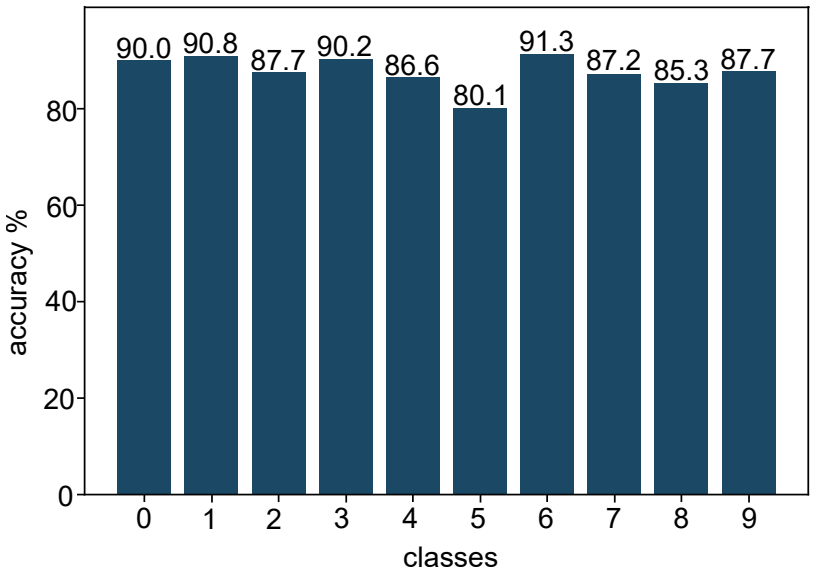

Figure S5. Test accuracy for each class.

We also found that training for longer epochs and fine tuning parameters help to bring $\sim 4\%$ improvement in accuracy. Figure S6 shows the training accuracy evolution for the versions that give test accuracy of 87.19% (before parameter fine tuning) and 87.78% (after parameter fine tuning).

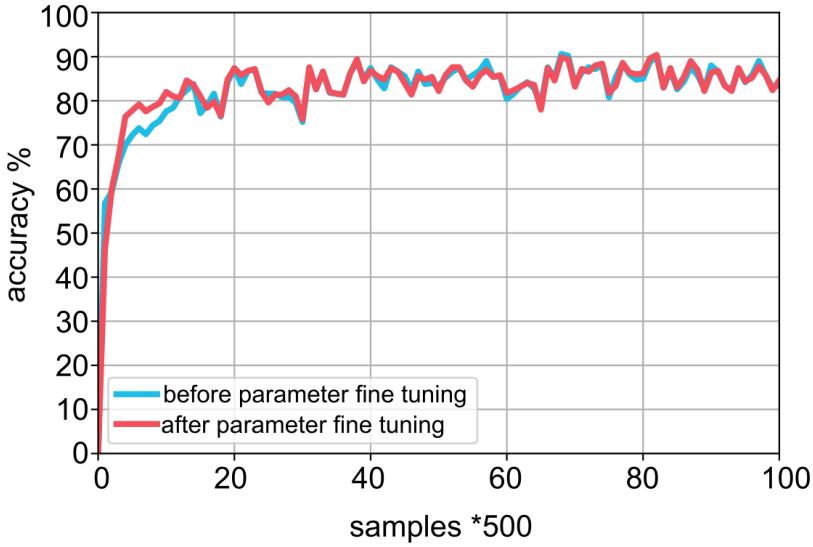

Figure S6. Training accuracy for running 50k epochs. Figure is plotted every 500 samples for clarity.